

SYDE 575 Lab 1



Robson Lamond (20991058)

Izaac Marvin Laundry-Mottiar (21017776)

Date September 27th, 2026

SYDE 575

Professor: David Clausi

Introduction

This lab explores basic image processing techniques, including digital zooming and point operations for image enhancement. First, three interpolation methods are compared using visual observations and quantitative PSNR calculations. Next, the histogram of an image is explored and the image is transformed using gamma correction and histogram equalization to determine the effects on image quality.

Procedure, Results, & Discussion

Digital Zooming

The images of Lena and Cameraman were first converted to greyscale and then down sampled by a factor of 4 in each direction using bilinear interpolation. The down sampled images are shown below.

Three different digital zooming techniques (Nearest Neighbor Interpolation, Bilinear Interpolation, and Bicubic Interpolation) were used to increase the resolution of the down-sampled images back to the resolution of the original images. The up-sampled images are shown below.



Figure 1: Digital Zooming Techniques on Cameraman Image

Lena: Original and Upsampled Images



Figure 2: Digital Zooming Techniques on Lena Image

1. What can you observed about the up-sampled images produced by each of the methods?

Nearest-neighbor interpolation looks grainy and blocky. Bilinear and bicubic interpolation look smoother, but some details are blurred. The darkest and brightest regions also appear slightly muted in the interpolated images.

2. How do the different methods compare to each other in terms of PSNR as well as visual quality? Why?

Nearest-neighbor has the lowest PSNR, followed by bilinear and then bicubic interpolation. Visual quality improves in the same order. Bicubic uses more nearby pixel information when estimating new pixels, so it usually produces an image closer to the original. The PSNR increases due to the less dramatic change between pixels in the original image and interpolated pixels in the up-sampled image as compared to the binary 'this or that' that is seen in nearest-neighbor. This is especially seen in high contrast areas.

3. What parts of the image seem to work well using these digital zooming methods? What parts of the image don't? Why?

High-contrast edges and large smooth regions work fairly well because their general shape is preserved. Fine textures and small details do not restore well because information is lost during down-sampling. For Cameraman, this is noticeable in the face, camera, and background structures. For Lena, this is noticeable in the hair, feathers, and hat.

4. Compare the zooming results between Lena and Cameraman. Which image results in higher PSNR? Which image looks better when restored to the original resolution using digital zooming methods? Why?

Lena has a higher PSNR for all zooming methods and generally looks better after restoration. Lena starts at 512×512 pixels, while Cameraman starts at 256×256 pixels. After down-sampling, Lena is 128×128 and Cameraman is 64×64 , so Lena retains four times as much pixel data. This extra data allows for more accurate reconstruction upon up sampling, which is mathematically shown with the higher PSNR values.

5. What does the PSNR tell you about each of the methods? Does it reflect what is observed visually?

PSNR gives a pixel-wise measurement of how closely two images match. Based on the results this tells us that the Bicubic method produces an image that has the least error when compared to the original image. On a relative scale, the difference in PSNR is higher between bicubic and bilinear than bilinear and nearest neighbor for both the Lena and Cameraman images. The higher scoring methods on PSNR also had better visual quality, matching the previous observations.

Point Operations

The tire image was first scaled down from $[0, 255]$ to $[0, 1]$ and a histogram was taken to inspect the original image. The image and histogram is shown below.

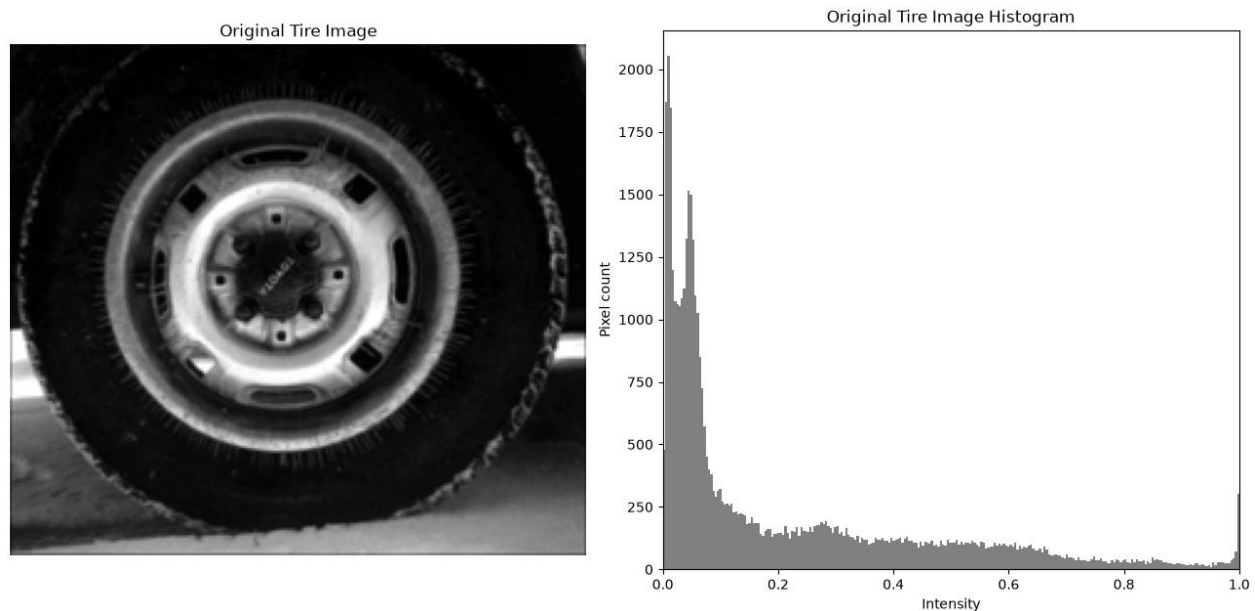


Figure 3: Original Tire Image and Histogram

6. Explain what the histogram of an image represents. Why is it useful?

The histogram of an image shows the frequency that a certain pixel intensity occurs in an image. The full scale of pixel intensities is divided into bins and the pixels of the image are sorted into these bins. The histogram is useful because it allows visual inspection of the pixel intensities found in the image. It can show how strong the highlights, shadows, or contrast is in an image.

7. Describe how the histogram looks like in the context of intensity distribution. What does the histogram say about the image

The histogram is concentrated at low, dark intensities. It has a large peak at the darkest pixels and a smaller peak below 0.1. There are relatively few mid-tone and light pixels, except for a small peak at the brightest values. This shows that the image is mostly dark with limited highlights.

Now, the negative transform is applied to the tire image which is shown below with its histogram.

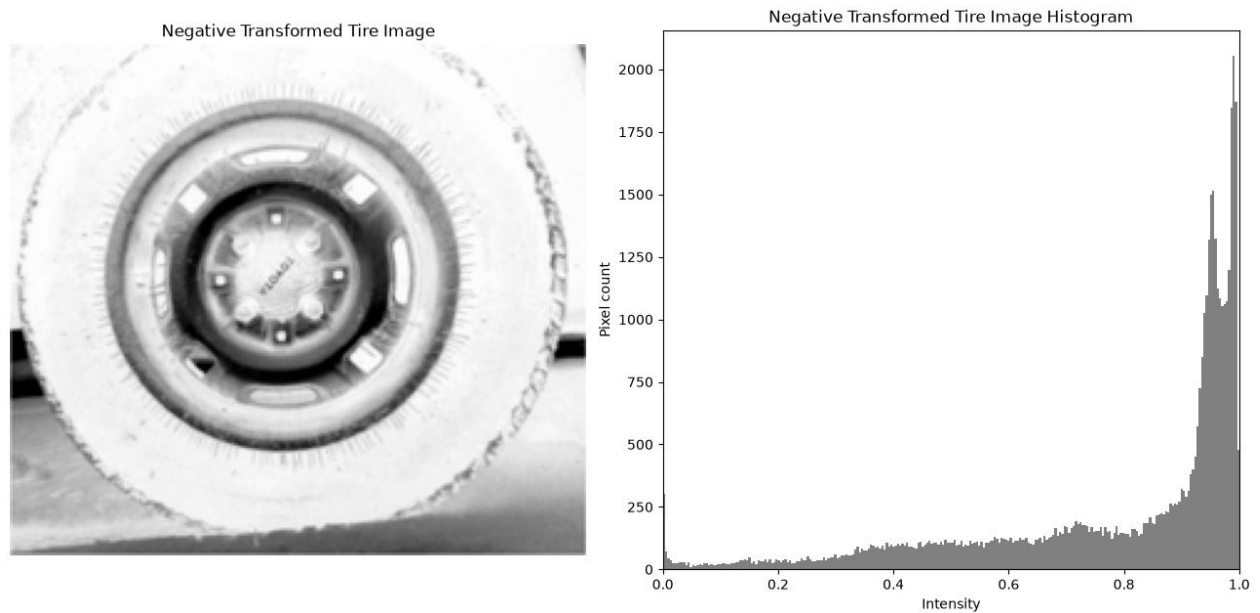


Figure 4: Negative Transformed Tire Image and Histogram

8. Describe how the histogram looks like in the context of intensity distribution. How does it differ from the histogram of the original image? Why?

The negative-image histogram is a mirror of the original histogram. The high number of dark pixels becomes a high number of bright pixels because the negative transform reverses each intensity value.

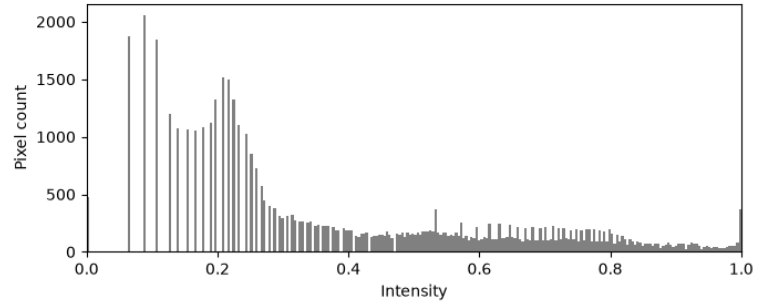
Next, a power law transformation was applied to the original tire image using a γ of 0.5 and a γ of 1.3. The transformed images and their respective histograms are shown below.

Power-law Transformed Tire Images

Power-law transform ($\gamma = 0.5$)



Histogram ($\gamma = 0.5$)



Power-law transform ($\gamma = 1.3$)



Histogram ($\gamma = 1.3$)

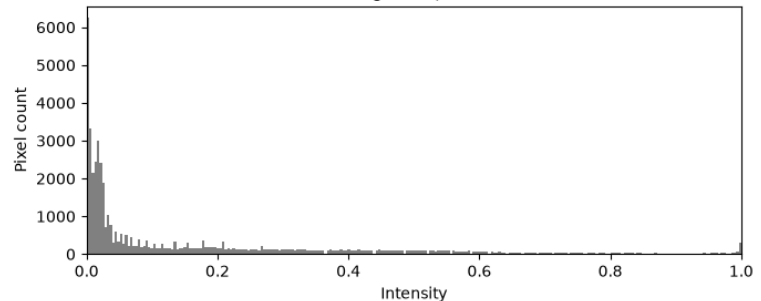


Figure 5: Power-law Transformed Tire Images and Histograms

9. Describe the appearance of the transformed images. Why do they appear this way?

The $\gamma=0.5$ image appears brighter than the original, while the $\gamma=1.3$ image appears darker. When γ is less than 1, lower intensity values are stretched and higher values are compressed, causing a brighter appearance. When γ is greater than 1, lower values are compressed and higher values are stretched, making the image darker.

10. Describe how each of the histograms look like in the context of intensity distribution. Why do they look like this? What does each histogram say about each transformed image?

The $\gamma=0.5$ histogram is spread more broadly across the intensity range because the many dark pixels are shifted toward brighter values. This produces a brighter image with a more even distribution of light and dark pixels. The $\gamma=1.3$ histogram is extremely concentrated at lower pixel intensities, so the image appears darker and has less visible detail than the original image.

11. Compared with the original image, which of the transforms should you use to enhance the image? Why?

The $\gamma=0.5$ transformation is the best choice for enhancing the image. Since the original image contains a high concentration of low intensity pixels and an “ideal” image has a flat histogram, applying this transformation distributes the low intensity pixels to a higher intensity range. This transformed image has a histogram closer to the “ideal” flat histogram. Visually, this image allows for more detail to be seen in the tire’s texture as well as the center of the tire.

Lastly, histogram equalization is performed on the original tire image. The transformed image and its histogram is shown below.

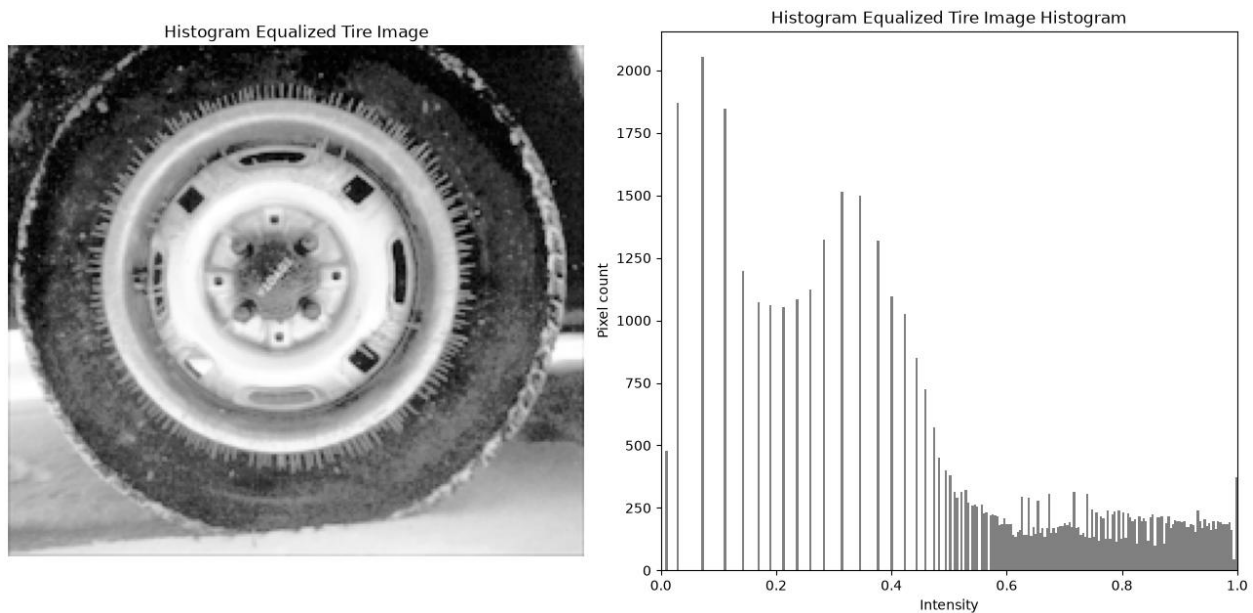


Figure 6: Histogram Equalized Tire Image and Histogram

12. Describe the appearance of the equalized image.

The histogram-equalized image has stronger contrast and more visible detail. It has a more balanced mix of dark and bright regions than the original image which makes it seem like there is more detail.

13. Describe how the histogram looks like in the context of its intensity distribution. Why does it look like this? What does the histogram say about the equalized image?

The equalized histogram is spread more evenly across the intensity range than any of the previous histograms analyzed. This is because histogram equalization uses the image's cumulative distribution function to remap pixel intensities and stretch concentrated ranges. This improves contrast and what is perceived as detail since the image makes better use of the available intensity range.

Summary

In this lab, different interpolation methods are used to restore down-sampled images. Nearest neighbor zooming produces the lowest PSNR values and appears blocky, where bilinear interpolation produces a smoother image. Bicubic interpolation gives the highest PSNR values and generally produced images closest to the original image with some blurring effect at high contrast edges.

The point operations explored show how intensity transformations affect image brightness and contrast. The $\gamma=0.5$ transformation brightens the tire image and makes details in darker areas easy to see. The $\gamma=1.3$ transformation darkens the image and concentrates a significant number of pixels at low intensities. Histogram equalization produces the greatest overall contrast, by spreading pixel intensities as evenly as possible across the full range. Overall, this lab shows the effect of interpolation and point operations on images and provides insight into applications of these techniques based on desired results.

Appendix

PSNR Function (psnr.py)

```
import numpy as np
from PIL import Image

def PSNR(
    reference_img: Image.Image | np.ndarray, test_img: Image.Image | np.ndarray
) -> float:
    if reference_img.size != test_img.size:
        raise ValueError("Input images must have the same dimensions.")

    MAX_PIXEL_VALUE = 255.0

    # Convert images to numpy arrays of type float64
    reference_array = np.array(reference_img, dtype=np.float64)
    test_array = np.array(test_img, dtype=np.float64)
```

```

# Calculate MSE
mse = np.mean((reference_array - test_array) ** 2)

# Calculate PSNR
if mse == 0:
    return float("inf")
psnr = 10 * np.log10((MAX_PIXEL_VALUE**2) / mse)

return psnr

```

Digital Zooming (run.py)

```

def step_1_digital_zooming():

    RESOLUTION_REDUCTION_FACTOR = 0.25

    # Load images
    cameraman = io.imread(CAMERAMAN_ASSET_PATH)
    lena = io.imread(LENA_ASSET_PATH)

    # Get image dimensions
    CAMERAMAN_WIDTH, CAMERAMAN_HEIGHT = cameraman.shape[1], cameraman.shape[0]
    LENA_WIDTH, LENA_HEIGHT = lena.shape[1], lena.shape[0]

    # Convert images to greyscale and float64 for processing
    cameraman_grey = np.array(cameraman, dtype=np.float64)
    lena_grey = color.rgb2gray(np.array(lena)) * 255

    # Downsample images
    reduced_cameraman = transform.resize(
        cameraman_grey,
        (
            int(CAMERAMAN_HEIGHT * RESOLUTION_REDUCTION_FACTOR),
            int(CAMERAMAN_WIDTH * RESOLUTION_REDUCTION_FACTOR),
        ),
        order=1,
        preserve_range=True,
    )
    reduced_lena = transform.resize(
        lena_grey,
        (
            int(LENA_HEIGHT * RESOLUTION_REDUCTION_FACTOR),
            int(LENA_WIDTH * RESOLUTION_REDUCTION_FACTOR),
        ),
        order=1,
        preserve_range=True,
    )

    # Upsample images using different interpolation methods
    upsampled_cameraman_nn = transform.resize(
        reduced_cameraman,
        (CAMERAMAN_HEIGHT, CAMERAMAN_WIDTH),
        order=0,
        preserve_range=True,
    )
    upsampled_lena_nn = transform.resize(
        reduced_lena, (LENA_HEIGHT, LENA_WIDTH), order=0, preserve_range=True
    )

```

```

upsampled_cameraman_bilinear = transform.resize(
    reduced_cameraman,
    (CAMERAMAN_HEIGHT, CAMERAMAN_WIDTH),
    order=1,
    preserve_range=True,
)
upsampled_lena_bilinear = transform.resize(
    reduced_lena, (LENA_HEIGHT, LENA_WIDTH), order=1, preserve_range=True
)

upsampled_cameraman_bicubic = transform.resize(
    reduced_cameraman,
    (CAMERAMAN_HEIGHT, CAMERAMAN_WIDTH),
    order=3,
    preserve_range=True,
)
upsampled_lena_bicubic = transform.resize(
    reduced_lena, (LENA_HEIGHT, LENA_WIDTH), order=3, preserve_range=True
)

# Calculate PSNR values for each upsampling method
cameraman_psnr_nn = PSNR(cameraman_grey, upsampled_cameraman_nn)
cameraman_psnr_bilinear = PSNR(cameraman_grey, upsampled_cameraman_bilinear)
cameraman_psnr_bicubic = PSNR(cameraman_grey, upsampled_cameraman_bicubic)
lena_psnr_nn = PSNR(lena_grey, upsampled_lena_nn)
lena_psnr_bilinear = PSNR(lena_grey, upsampled_lena_bilinear)
lena_psnr_bicubic = PSNR(lena_grey, upsampled_lena_bicubic)

show_upsampling_comparison(
    original_image=cameraman_grey,
    upsampled_images=[
        upsampled_cameraman_nn,
        upsampled_cameraman_bilinear,
        upsampled_cameraman_bicubic,
    ],
    image_name="Cameraman",
    psnr_values=[
        cameraman_psnr_nn,
        cameraman_psnr_bilinear,
        cameraman_psnr_bicubic,
    ],
)
show_upsampling_comparison(
    original_image=lena_grey,
    upsampled_images=[
        upsampled_lena_nn,
        upsampled_lena_bilinear,
        upsampled_lena_bicubic,
    ],
    image_name="Lena",
    psnr_values=[lena_psnr_nn, lena_psnr_bilinear, lena_psnr_bicubic],
)

```

Point Operations (run.py)

```
def step_2_tire_image_and_histogram():

    # Load tire image
    tire = io.imread(TIRE_ASSET_PATH)

    # Scale the tire image to the range [0, 1]
    scaled_tire = tire / 255.0

    show_image_and_histogram(
        image=scaled_tire, title="Original Tire Image", intensity_range=(0, 1)
    )

def step_3_negative_transform():

    # Load tire image
    tire = io.imread(TIRE_ASSET_PATH)

    # Scale the tire image to the range [0, 1]
    scaled_tire = tire / 255.0

    # Apply negative transformation
    negative_transformed_tire = 1 - scaled_tire

    show_image_and_histogram(
        image=negative_transformed_tire,
        title="Negative Transformed Tire Image",
        intensity_range=(0, 1),
    )

def step_4_power_law_transformations():

    # Load tire image
    tire = io.imread(TIRE_ASSET_PATH)

    # Scale the tire image to the range [0, 1]
    scaled_tire = tire / 255.0

    power_law_transformed_tire_05 = exposure.adjust_gamma(scaled_tire, gamma=0.5)
    power_law_transformed_tire_13 = exposure.adjust_gamma(scaled_tire, gamma=1.3)

    show_power_law_transformations(
        transformed_images=[
            power_law_transformed_tire_05,
            power_law_transformed_tire_13,
        ],
        gammas=[0.5, 1.3],
    )

def step_5_histogram_equalization():

    # Load tire image
    tire = io.imread(TIRE_ASSET_PATH)

    # Scale the tire image to the range [0, 1]
    scaled_tire = tire / 255.0

    # Apply histogram equalization
    equalized_tire = exposure.equalize_hist(scaled_tire)

    show_image_and_histogram(
        image=equalized_tire,
        title="Histogram Equalized Tire Image",
        intensity_range=(0, 1),
    )
```

)

Helper Functions

Note that GenAI was used in writing the following plotting helper functions for displaying images for the report.

```
def show_upsampling_comparison(
    original_image: np.ndarray,
    upsampled_images: list[np.ndarray],
    image_name: str,
    psnr_values: list[float],
) -> None:
    methods = ["Original", "Nearest Neighbor", "Bilinear", "Bicubic"]
    figure, axes = plt.subplots(1, len(methods), figsize=(14, 4), constrained_layout=True)

    images = [original_image, *upsampled_images]
    for column, (image, method) in enumerate(zip(images, methods)):
        axes[column].imshow(image, cmap="gray", vmin=0, vmax=255)
        title = (
            method
            if column == 0
            else f"{method}\nPSNR: {psnr_values[column - 1]:.2f} dB"
        )
        axes[column].set_title(title)
        axes[column].axis("off")

    figure.suptitle(f"{image_name}: Original and Upsampled Images", fontsize=16)
    plt.show()

def show_image_and_histogram(
    image: np.ndarray,
    title: str,
    intensity_range: tuple[float, float] | None = None,
) -> None:
    """Display a grayscale image beside its 256-bin intensity histogram."""
    if intensity_range is None:
        intensity_range = (float(image.min()), float(image.max()))

    figure, (image_axis, histogram_axis) = plt.subplots(
        1, 2, figsize=(10, 4), constrained_layout=True
    )
    image_axis.imshow(
        image,
        cmap="gray",
        vmin=intensity_range[0],
        vmax=intensity_range[1],
    )
    image_axis.set_title(title)
    image_axis.axis("off")

    histogram_axis.hist(image.ravel(), bins=256, range=intensity_range, color="gray")
    histogram_axis.set_title(f"{title} Histogram")
    histogram_axis.set_xlabel("Intensity")
    histogram_axis.set_ylabel("Pixel count")
    histogram_axis.set_xlim(intensity_range)
    plt.show()

def show_power_law_transformations(
    transformed_images: list[np.ndarray], gammas: list[float]
) -> None:
```

```

"""Display each power-law transformed image beside its intensity histogram."""
figure, axes = plt.subplots(
    len(transformed_images), 2, figsize=(10, 8), constrained_layout=True
)
axes = np.atleast_2d(axes)

for row, (image, gamma) in enumerate(zip(transformed_images, gammas)):
    image_axis, histogram_axis = axes[row]
    image_axis.imshow(image, cmap="gray", vmin=0, vmax=1)
    image_axis.set_title(f"Power-law transform ( $\gamma = \{gamma\}$ ")
    image_axis.axis("off")

    histogram_axis.hist(image.ravel(), bins=256, range=(0, 1), color="gray")
    histogram_axis.set_title(f"Histogram ( $\gamma = \{gamma\}$ ")
    histogram_axis.set_xlabel("Intensity")
    histogram_axis.set_ylabel("Pixel count")
    histogram_axis.set_xlim(0, 1)

figure.suptitle("Power-law Transformed Tire Images", fontsize=16)
plt.show()

```